

LLM Scaling Law: 从经验幂律、Compute-Optimal 到数据与推理扩展

一份面向预训练实践的技术认知与判断指南

Jiguo Li¹

调研与修订日期：2026 年 8 月 7 日

摘要

LLM scaling law 不是“参数越大、能力越强”的口号，而是在固定数据分布、模型族与训练配方下，用一组受控的小规模实验刻画损失随参数量、训练 token 与计算量变化的局部经验规律，并据此进行资源配置和外推决策。本文沿着 learning curve、Kaplan 三轴幂律、Chinchilla compute-optimal、data-constrained/data-quality scaling、MoE 与 inference-time scaling 的脉络，给出核心公式、推导、拟合和验证细节。报告进一步整理 2020–2026 年 28 个公开模型家族的尺寸、训练 token、total/active parameters 与证据等级，用来检验公开实践支持什么、不能支持什么。面向预训练工程，本文给出 isoFLOP 实验矩阵、鲁棒拟合、scale holdout、bootstrap、不确定性传播，以及 code/repo-level 数据策略的可量化验证框架。核心结论是：同分布 validation loss 的局部外推最可靠；跨代 benchmark 与能力涌现最不可靠；函数形式可以迁移，系数、最优 token/parameter 比和能力阈值通常不能直接迁移。

关键词：大语言模型；Scaling Law；Compute-Optimal；数据扩展；MoE；推理时计算；代码预训练

核心判断

Scaling law 是局部经验模型、受控实验方法与资源决策工具的组合。它最擅长回答“在相同 recipe 下，扩大多少参数或 token 能降低多少 loss”，而不是回答“某种智能何时必然出现”。

¹ 本文在 Codex 协助下完成

目录

1	引言：先分清四类 Scaling 问题	3
2	问题起源：从 Learning Curve 到语言模型幂律	3
2.1	早期起点：误差随数据量呈幂律下降	3
2.2	Kaplan：统一参数、数据和计算三轴	3
2.3	为什么幂律有用，却不是自然定律	4
3	核心数学：从联合损失面到 Compute-Optimal	4
3.1	变量口径与训练计算近似	4
3.2	Chinchilla 联合损失模型及推导	4
3.3	Kaplan 与 Chinchilla 的差异	4
4	技术演进： Scaling 的自变量不断增加	5
4.1	迁移、多模态与生产级预测	5
4.2	从平滑 Loss 到“涌现”争论	5
4.3	从“更多数据”到“有效数据”	5
4.4	从训练最优到全生命周期最优	5
5	实现细节：怎样做一套可用于发车决策的 Scaling Study	5
5.1	先定义决策，再定义实验	5
5.2	最小实验矩阵与控制变量	6
5.3	鲁棒拟合、外推与不确定性	6
5.4	三种方法互相校验	7
6	公开 LLM 尺寸：实践如何印证、又如何混淆 Scaling	7
6.1	MoE：Total Parameters 与 Active Parameters 是两条轴	7
7	对 Code Pretraining 与数据 Pipeline 的直接启示	8
7.1	不要只拟合 Aggregate Loss	8
7.2	比较数据策略时必须等价的 Setting	8
7.3	Repo-Level 组织带来的特殊混杂	8
8	判断框架： Scaling Law 能预测什么、不能预测什么	8
9	总结与展望	9
A	术语速查	10
B	公开模型尺寸与 Scaling 证据清单	10
C	建议的数据记录 Schema 与发车清单	12
	参考文献	13

1 引言：先分清四类 Scaling 问题

大模型讨论中，至少有四类问题经常被混在一起。第一类是 **loss scaling**：模型参数量 N 、训练 token 数 D 或训练计算量 C 增长时，held-out cross-entropy loss 是否在有限尺度区间内平滑下降。第二类是 **compute-optimal scaling**：给定训练预算，应把计算分给更大的模型还是更多的数据。第三类是 **capability scaling**：连续的 loss 改善是否会变成连续的 MMLU、代码生成或推理能力。第四类是 **system/economic scaling**：训练 FLOP-optimal 是否也是推理和部署成本最优。

这四个问题的证据强度不同。连续、同分布且与训练目标一致的 validation loss 通常噪声最低；exact match、pass@1 等离散指标受到阈值、prompt、采样和污染影响；全生命周期最优又取决于调用量、显存、延迟和服务生命周期。因此，本文采用表 1 的证据分级。

表 1 Scaling 证据分级。只有 A 级证据适合拟合规律，B/C 级主要用于趋势校验。

等级	定义	典型材料
A	同架构、同 tokenizer、同数据分布，系统改变 $N/D/C$ ，公开 loss 或 checkpoints	Chinchilla pilot、Pythia、Cerebras-GPT
B	同代模型家族，关键训练信息公开，但尺寸点较少或 token/recipe 随尺寸改变	GPT-3、LLaMA、Llama 2/3.1、Qwen2.5
C	跨代或单 endpoint，架构、数据、多模态或 post-training 改动显著	Gemma 2/4、Llama 4、DeepSeek-V4

证据边界

公开模型尺寸表不是一条 universal scaling curve。不同 tokenizer、数据质量、代码比例、架构、上下文长度和 post-training 都是强混杂变量；跨家族散点最多说明工程趋势，不能识别参数量的因果效应。

2 问题起源：从 Learning Curve 到语言模型幂律

2.1 早期起点：误差随数据量呈幂律下降

机器学习长期使用 learning curve 观察数据量与泛化误差的关系。现代神经 scaling law 的转折点是 Hestness 等人在机器翻译、语言建模、图像和语音等任务上观察到：在多个数量级内，可约误差常近似服从简单幂律，最优模型规模也随数据量规律增长 [1]。Rosenfeld 等人进一步提出同时依赖模型规模和数据规模的函数形式，并用较小尺度实验预测更大尺度表现 [2]。

单变量幂律通常写为

$$L(x) = L_{\infty} + Ax^{-\alpha}, \quad (1)$$

其中 x 可以是参数量、数据量或计算量； L_{∞} 是当前分布与模型族下的渐近损失下限； $Ax^{-\alpha}$ 是可通过扩展资源降低的部分； $\alpha > 0$ 是扩展效率。减去 L_{∞} 后，式 (1) 在 log-log 坐标中成为直线：

$$\log(L - L_{\infty}) = \log A - \alpha \log x. \quad (2)$$

指数通常很小，意味着边际收益递减很强。更重要的是， L_{∞} 未知时不能直接做普通线性回归；否则斜率会产生系统偏差。

2.2 Kaplan：统一参数、数据和计算三轴

Kaplan 等人把语言模型的参数量 N 、训练 token 数 D 与训练计算量 C 放入统一框架，跨多个数量级拟合 loss 曲线 [3]。其代表性指数约为 $\alpha_N = 0.076$ 、 $\alpha_D = 0.095$ 、 $\alpha_C = 0.050$ ，并得到早期 compute-optimal 结论：预算增长时模型规模应比数据量增长得更快，近似为

$$N_{\text{opt}} \propto C^{0.73}, \quad D_{\text{opt}} \propto C^{0.27}. \quad (3)$$

这一结论影响了 GPT-3 式“大模型、相对少 token、较早停止”的训练路线 [6]。它的贡献不在于给出永恒指数，而在于把“扩大模型是否有效”变成了“可用 pilot 预测目标 run”的工程问题。

2.3 为什么幂律有用，却不是自然定律

幂律可以来自不同 regime 下的方差限制、分辨率限制或逼近误差；理论工作能够解释常见的函数形态，但仍不能给出跨数据分布与架构通用的指数 [4]。经验上还会出现 plateau、double descent、迟发跃迁或 slope change, smoothly broken power law 往往比单一幂律更合适 [5]。因此需要把式 (1) 理解为有限区间、固定 recipe 下的局部模型。

3 核心数学：从联合损失面到 Compute-Optimal

3.1 变量口径与训练计算近似

表 2 给出本文统一口径。对于标准 dense decoder-only Transformer，常用一阶近似

$$C \approx 6ND. \quad (4)$$

这里 forward 约为 $2ND$ FLOPs, backward 约为 forward 的两倍。该式只适合粗略规划：embedding、长上下文 attention 的 $O(S^2)$ 项、MoE routing、激活专家数和通信都会使其失真。

表 2 Scaling study 的核心变量与工程口径。

符号	含义	实践口径
N	模型规模	dense 常用 non-embedding parameters; MoE 另记 total/active
D	训练数据量	实际送入优化器的 token; 重复数据同样计数
C	训练计算量	theoretical FLOPs、device FLOPs 或 wall-clock 应分开记录
L	held-out loss	固定 tokenizer、固定验证分布上的 token-level NLL
E	irreducible loss	需要拟合的渐近项，不应随意固定为论文数字

3.2 Chinchilla 联合损失模型及推导

Hoffmann 等人训练了 400 多个、70M–16B 参数、5B–500B token 的模型，采用联合损失面 [7]：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad A, B, \alpha, \beta > 0. \quad (5)$$

在式 (4) 的计算约束下，令 $D = C/(6N)$ ，代回式 (5)：

$$L(N | C) = E + AN^{-\alpha} + B \left(\frac{6N}{C} \right)^\beta. \quad (6)$$

令式 (6) 对 N 的导数为零，可得两个边际收益平衡，并得到

$$N_{\text{opt}}(C) = G \left(\frac{C}{6} \right)^{\frac{\beta}{\alpha+\beta}}, \quad G = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}}, \quad (7)$$

$$D_{\text{opt}}(C) = G^{-1} \left(\frac{C}{6} \right)^{\frac{\alpha}{\alpha+\beta}}. \quad (8)$$

若记 $a = \beta/(\alpha + \beta)$ 、 $b = \alpha/(\alpha + \beta)$ ，则 $a + b = 1$ 。直觉上，数据项的边际收益越高，最优点越向数据侧移动；模型项的边际收益越高，最优点越向参数侧移动。

3.3 Kaplan 与 Chinchilla 的差异

Chinchilla 以与 Gopher 近似的训练计算量，把 280B 参数、300B token 改为 70B 参数、1.4T token，并在其报告的多数任务上更好 [8, 7]。这使 scaling law 从“解释曲线”变成了“直接改变训练配方”的工具。

“约 20 tokens/parameter”只是特定数据、模型族和实验区间下的工程口径。它优化单次训练 FLOPs，不包含推理成本、显存、数据质量与重复。Besiroglu 等人的复现还发现 Chinchilla Approach 3 的拟合存在优化器提前停止和置信区间过窄问题；重新拟合的点估计仍与约 20–26 tokens/parameter 相容，但合理区间明显更宽 [9]。稳健结论是参数和 token 应共同扩展，而不是精确常数可跨体系迁移。

表 3 两代 compute-optimal 结论的差异。

比较项	Kaplan 2020	Chinchilla 2022
N_{opt} 对 C 的指数	0.73	约 0.49–0.50
D_{opt} 对 C 的指数	0.27	约 0.50–0.51
工程含义	更快增大模型、较早停止	参数与 token 近似同比例增长
代表实践	GPT-3 175B / 300B	Chinchilla 70B / 1.4T

4 技术演进：Scaling 的自变量不断增加

4.1 迁移、多模态与生产级预测

Henighan 等人把幂律扩展到图像、视频、多模态和数学问题生成 [10]；Hernandez 等人把预训练收益解释为下游数据的“有效倍增”，并刻画 transfer scaling [11]。GPT-4 技术报告展示了生产级用途：用计算量低至四个数量级的小模型预测最终内部 codebase loss，并以低至三个数量级的模型预测 HumanEval 子集表现；报告同时明确部分能力仍难预测 [12]。这支持的是受控训练体系内的可预测性，不是任意能力的普适预测。

4.2 从平滑 Loss 到“涌现”争论

“Emergent abilities”把小模型近乎为零、大模型突然上升的 benchmark 能力解释为跃迁 [13]。但不连续指标本身就能制造跳变：将 exact match 换为 token-level accuracy 或 Brier score 后，很多曲线会更平滑 [14]。因此至少要区分：模型内部能力可能平滑增长；任务成功率可能因阈值呈 S 型；prompt 与采样制造高方差；三个模型点不足以证明 phase transition。

4.3 从“更多数据”到“有效数据”

Data-constrained scaling 研究表明，在固定计算预算下适度重复数据可能损失很小，但重复的边际价值最终趋近于零 [15]。DeepSeek LLM 报告不同数据版本会改变最优分配指数，说明 scaling coefficient 本身可作为数据与 recipe 的体系内诊断量 [16]。DataComp-LM 用标准化小模型实验展示过滤策略能显著移动性能前沿 [17]；data mixing law 则把 mixture proportion 也作为可拟合变量 [18]。

把这些结果统一起来，可以写成更现实但不可直接一次拟合的关系：

$$\text{quality} = f(N, D, Q_{\text{data}}, C_{\text{train}}, C_{\text{test}}, \text{architecture}, \text{objective}). \quad (9)$$

这里 Q_{data} 不是已解决的单一标量，而是 filtering、dedup、domain mix、污染、信息密度与重复结构的集合。

4.4 从训练最优到全生命周期最优

若模型会被调用 R 次，决策目标更接近

$$J = C_{\text{train}} + R c_{\text{infer}}(N, C_{\text{test}}), \quad (10)$$

而不是只最小化训练 FLOPs。Inference-aware scaling 的结果是：高调用量下，训练一个更小但看过更多 token 的模型可能拥有更低总成本 [19]。Test-time scaling 进一步表明，自适应分配搜索或 verifier compute，在部分问题上可以比单纯扩大参数更有效 [20]。现代 scaling 因而从 N - D - C_{train} 三角形扩展到训练、部署与推理时计算的联合优化。

5 实现细节：怎样做一套可用于发车决策的 Scaling Study

5.1 先定义决策，再定义实验

一个可用研究必须回答可执行问题，例如：给定 10^{22} FLOPs，选 1B/20B tokens 还是 2B/10B tokens；新的 code filter 是否能在等 FLOPs 下移动 OOD code loss frontier；8K repo-level packing 的收益能否从 300M/1B pilot 外推到 7B；或服务量达到多少时 longer-trained 7B 优于 13B。没有决策目标，最终通常只有好看的拟合曲线，没有训练配方。

5.2 最小实验矩阵与控制变量

表 4 是面向 7B 目标模型的最小可用设计。规模比绝对数更重要：每条 isoFLOP 至少覆盖最优点两侧，最大 pilot 与目标点之间不宜跨越多数量级。

表 4 建议的 pilot 实验矩阵。每个格点代表一个完整 run，不是同一 run 的 checkpoint。

维度	推荐取值	目的	最低控制要求
模型规模 N	150M, 300M, 600M, 1.2B, 2.4B	覆盖约 1 个数量级	相同 depth/width 规则与架构族
IsoFLOP C	4-6 个计算档	找到每档 U 型 loss- N 曲线	每档至少 4 个 N/D 组合
数据版本	baseline/candidate	判断数据策略是否移动 frontier	同 raw snapshot、tokenizer 与 mix
随机性	关键点 2-3 seeds	估计 run variance	相同初始化分布和数据顺序规则
验证集	in-domain + OOD + code/repo	区分拟合与迁移	时间切分、去污染、固定版本

每个 run 至少记录：模型总参数、non-embedding 参数、MoE active 参数、实际 token、unique token、epoch、理论 FLOPs、实测吞吐、序列长度、tokenizer、数据版本、语言/code mix、optimizer、LR schedule、seed、checkpoint loss 与硬件异常。重复 token 和 unique token 必须分开；MoE 的 total/active parameters 必须分开。

5.3 鲁棒拟合、外推与不确定性

对观测点 i ，可以拟合 log-space 残差

$$r_i = \log(E + AN_i^{-\alpha} + BD_i^{-\beta}) - \log L_i, \quad (11)$$

并使用 Huber loss。推荐做法是对 A, B, E 使用对数参数化，给 α, β 正值边界，多组初始化，保留并标注失败点；bootstrap 必须按 run 重采样，不能把同一 run 的高度相关 checkpoints 当成独立样本。验证集应整条留出更高 compute frontier，不能只随机拆分观测点。

代码 1: Chinchilla 风格联合损失面的最小拟合骨架。

```
import numpy as np
from scipy.optimize import least_squares

def residual(theta, n, d, observed_loss):
    log_a, log_b, log_e, alpha, beta = theta
    pred = (np.exp(log_e)
            + np.exp(log_a) * n ** (-alpha)
            + np.exp(log_b) * d ** (-beta))
    return np.log(pred) - np.log(observed_loss)

starts = [[6.0, 6.0, 0.5, a, b]
           for a in (0.15, 0.30, 0.50, 0.80)
           for b in (0.15, 0.30, 0.50, 0.80)]
fits = [least_squares(
    residual, x0=s, args=(n, d, loss),
    bounds=([-20, -20, -20, 0.01, 0.01],
            [40, 40, 5, 2.00, 2.00]),
    loss="huber", f_scale=1e-3)
    for s in starts]
fit = min(fits, key=lambda x: np.sum(x.fun ** 2))
```

表 5 Compute-optimal 的三种估计方法。

方法	做法	优点	主要风险
Training-curve minimum	每个 N 跑足，找给定 compute 的最低 loss	直观	需要密集 checkpoint，早停误差大
IsoFLOP profile	固定多个 C ，扫描不同 N/D	直接回答资源分配	frontier 点少时插值敏感
Parametric surface	拟合 $L(N, D)$ 后解析求最优	利用全部点并连续外推	依赖函数形式、初值和异常点

5.4 三种方法互相校验

三种方法在目标尺度附近一致，且 scale holdout 落入预测区间，才适合发车昂贵的大 run。必做诊断包括： $\log(L - E)$ 对 $\log N / \log D / \log C$ ；每条 isoFLOP 的 U 型曲线；residual heatmap；预测值对实际值； $N_{\text{opt}}(C)$ 与 $D_{\text{opt}}(C)$ 的 bootstrap band；目标 run 的 prediction interval。

当 residual 随 scale 呈系统趋势、最大尺度持续偏向同一方向、 E 贴边、数据或 tokenizer 已改变、dense 切换 MoE、context 增长导致 attention FLOPs 不可忽略时，应判定旧 law 失效并重新拟合。

6 公开 LLM 尺寸：实践如何印证、又如何混淆 Scaling

附录 B 收录 2020–2026 年 28 个公开模型家族。主文只抽取适合形成判断的对照。表 6 中，GPT-3、Pythia、Cerebras-GPT 尺寸点密集；LLaMA、Llama 3.1、Qwen2.5/Qwen3 与 OLMo 2 更能代表现代 data-rich 与 deployment-oriented 训练。

表 6 具有代表性的 dense 模型家族及其证据边界。

家族	尺寸梯度	训练 token	能支持的判断
GPT-3	125M–175B，共 8 点	每个约 300B	同 recipe 的 size/few-shot 趋势；不适合估计 compute-optimal ratio[6]
Pythia	70M–12B，共 8 点	每个约 300B	开放 checkpoints 与固定数据顺序，适合训练动力学 [21]
Cerebras-GPT	111M–13B，共 7 点	20 tokens/param.	公开 Chinchilla-style family[22]
LLaMA	7B, 13B, 33B, 65B	1.0T/1.4T	data-rich 小模型可超过旧大模型；token budget 不同 [23]
Llama 3.1	8B, 70B, 405B	超过 15T	现代家族远超 20 tokens/parameter；不可与旧代连线 [32]
Qwen2.5	0.5B–72B，共 7 点	18T	现代 dense 尺寸梯度，说明数据/recipe 移动 frontier[33]
Qwen3 Dense	0.6B–32B，共 6 点	约 36T	新代小模型追平旧代大模型是曲线整体移动，不是参数失效 [40]
OLMo 2	1B, 7B, 13B, 32B	最高 5T–6T	数据、代码、recipe 与 checkpoints 可审计 [39]

公开实践给出五条相对稳健的结论。第一，参数不是独立自变量；LLaMA-13B 超过 GPT-3 175B 的多项结果来自数据、token、架构和 recipe 共同移动曲线。第二，训练显著 data-rich：从 GPT-3 175B/300B，到 Chinchilla 70B/1.4T，再到 Llama 3.1 的 15T+ 与 Qwen3 的约 36T。第三，这不表示 Chinchilla“错误”，因为现代模型还优化推理成本、可部署性和模型家族复用。第四，过滤、混合与领域数据能够横向移动整条 frontier。第五，MoE 把“规模”拆成容量与计算。

6.1 MoE：Total Parameters 与 Active Parameters 是两条轴

表 7 展示如果只看 total parameters，会如何误读 MoE。总参数更接近可存储的专家容量，active parameters 更接近单 token 的前向计算，但 routing、共享专家、通信与负载均衡仍不可忽略。

表 7 代表性 MoE 模型。Total 与 active parameters 不能混为一谈。

模型	Total	Active/token	Tokens	正确解读
Mixtral 8x7B	46.7B	12.9B	未公开	存储容量近 47B，单 token 计算近 13B[29]
DeepSeek-V2	236B	21B	8.1T	大 total capacity 与低 active compute[37]
DeepSeek-V3	671B	37B	14.8T	不能将 671B 直接代入 dense 式 (4)[38]
Qwen3-30B-A3B	30B	3B	约 36T	小 active 模型借专家容量提升效果 [40]
Qwen3-235B-A22B	235B	22B	约 36T	total/active 共同决定容量与成本 [40]
Llama 4 Scout/Maverick	109B/400B	17B/17B	未公开	多模态与 distillation 混杂，不是 clean scaling suite[41]
DeepSeek-V4 Flash/Pro	284B/1.6T	13B/49B	未公开	2026 endpoint，只支持架构趋势观察 [44]

7 对 Code Pretraining 与数据 Pipeline 的直接启示

7.1 不要只拟合 Aggregate Loss

代码数据策略至少要分别拟合

$$L_{\text{general}}(N, D), \quad L_{\text{code}}(N, D), \quad L_{\text{repo}}(N, D), \quad L_{\text{swe}}(N, D). \quad (12)$$

同一数据改动可能降低 code loss，却提高 general loss；aggregate loss 会被占比最大的 web text 掩盖。loss frontier 与 HumanEval、MBPP、CrossCodeEval、SWE held-out 应分开报告，下游指标还要完成污染审计。

7.2 比较数据策略时必须等价的 Setting

对 baseline 与 candidate pipeline，应固定 raw snapshot、tokenizer、unique token budget、语言/code mix、 $N/D/C$ 、batch tokens 与 LR schedule；记录过滤保留率、dedup ratio、平均文档长度、code token ratio、repo coverage；同时评估 in-domain 与时间切分 OOD validation。核心指标不只包括最终 ΔL ，还应包括每 10^{20} FLOPs 的 loss 改善，以及达到目标 loss 所需 FLOPs。

若 candidate 用 D token 达到 baseline 用 kD token 才达到的 loss，则可定义 data multiplier

$$M_{\text{data}} = k. \quad (13)$$

它能把“loss 降低 0.01”翻译为 token efficiency 与算力收益，但前提是两个版本的 tokenizer、验证集与训练轨迹可比。

7.3 Repo-Level 组织带来的特殊混杂

repo/func-level packing 同时改变 token 邻接分布、cross-file mutual information、sequence utilization、样本长度、attention FLOPs、重复函数密度与污染传播范围。因此“训练 token 相同”仍不等价；还要报告 effective non-padding tokens、平均 attention length、每 token 实测 FLOPs，以及 file/repo 级 dedup 后的 unique coverage。

推荐两阶段闭环：第一阶段以 150M–2.4B 覆盖 4–6 条 isoFLOP，筛选 pipeline 并预测 7B；第二阶段至少做一个 7B 等级、等训练 token 的 baseline/candidate A/B run。发车门槛可设为：目标尺度 predicted code OOD loss 改善的 80% bootstrap lower bound 大于 0；general OOD loss 不越 guardrail；等 FLOPs 的 token efficiency 可量化；语言配比与污染无显著偏移；target confirmation 落在 pilot prediction interval 内。

8 判断框架：Scaling Law 能预测什么、不能预测什么

审阅任何 scaling claim 时，应依次问：横轴究竟是 total、non-embedding、active params、tokens 还是 FLOPs；纵轴是 pretraining loss、OOD loss 还是 benchmark；tokenizer、数据分布、架构和 optimizer 是否固定；覆盖多少数量级； E 如何拟合；是否有最大尺度 holdout；是否报告 seed variance、bootstrap interval 与 residual；是否区分

表 8 不同预测目标的相对可信度。

目标	可信度	主要原因
同 tokenizer、同分布 validation loss	高	连续、低噪声、与训练目标一致
邻近尺度 compute-optimal N/D	中高	需要足够 frontier 点和误差带
OOD domain loss	中	取决于 domain overlap 与 data mix
连续 downstream metric	中低	prompt、transfer 与 contamination 介入
Exact match/pass@1	低	离散阈值与高方差
新能力何时出现	很低	定义、指标、数据和 elicitation 均不稳定

unique/repeated tokens；污染、prompt 与评分器版本是否受控；优化目标究竟是训练 FLOPs、wall-clock、推理成本还是总拥有成本。

常见错误包括：把模型排行榜当 scaling law；把 20 tokens/parameter 当硬规范；比较不同 tokenizer 的 nats/token；随机拆 checkpoints 做验证；只报 R^2 ；以 total parameters 估 MoE FLOPs；把 benchmark 跳变直接解释为机制突变；静默删除失败 run；把论文系数直接搬到内部数据。通常可迁移的是函数形式与实验方法，不是系数。

9 总结与展望

LLM scaling law 的技术史可以概括为三次认知升级：Hestness 与 Kaplan 说明性能随资源增长具有可预测结构；Chinchilla 把问题从“是否变好”改为“参数与数据如何分配”；此后数据质量、重复、mixture、MoE active compute、推理时计算与部署经济性成为新的扩展轴。

对入门者，最重要的认知是：scaling law 不是智能增长的自然定律，而是固定体系内的局部经验模型。对预训练团队，最重要的动作是在自己的 tokenizer、数据版本、模型架构和 OOD validation 上做 pilot，用 isoFLOP 与 parametric surface 互相校验，报告外推区间，再用目标尺度 A/B run 闭环。公开模型实践提供方向性印证，但不能替代内部 scaling study。

仍待解决的问题包括：如何把数据质量压缩为可跨体系比较的变量；多域 mixture 如何随规模联合外推；合成数据的有效 token 如何计量；MoE 的容量、active compute、routing entropy 与通信如何统一；train-time 与 test-time compute 如何联合最优；token loss 如何映射到 repo-level editing 和 agent 长程任务；以及何时能在代价可控时提前检测 regime change。下一阶段更值得研究的不是更宏大的“万能曲线”，而是带证据等级、误差边界与决策闭环的局部 scaling laws。

A 术语速查

表 9 Scaling law 常用术语。

术语	定义
Scaling law	资源规模与 loss/性能之间的经验函数关系。
Power law	$y = Ax^{-\alpha}$ ；减去渐近项后在 log-log 坐标近似直线。
Reducible loss	可通过更多模型、数据或计算降低的 loss。
Irreducible loss	当前数据分布与模型族下的渐近下限。
Compute-optimal	固定训练计算预算时使目标 loss 最低的资源分配。
IsoFLOP	固定训练 FLOPs，扫描不同模型/数据组合。
Tokens/parameter	训练 token 数与参数量之比；不是跨 recipe 通用常数。
Active parameters	MoE 中每个 token 实际激活并参与计算的参数。
Data multiplier	candidate 数据达到同 loss 时相当于 baseline 的有效数据倍数。
Broken scaling law	允许不同 scale regime 中斜率平滑变化的经验规律。

B 公开模型尺寸与 Scaling 证据清单

表 10 的 token 数来自对应论文或官方发布页。“未公开”表示没有 fit-ready 的公开数字，并非推断为零；open weights 不等同于训练数据与过程完全开放。若 token budget 随尺寸变化，应回到原报告逐模型展开。2026 年模型只在官方资料明确给出尺寸时纳入，并统一标为 C 级趋势证据。

表 10 2020–2026 年公开 LLM 家族、尺寸、训练 token 与 scaling 证据。

年份	家族	架构	Total parameter sizes	Active/token	Pretraining tokens	证据	可支持的判断
2020	GPT-3[6]	Dense	125M; 350M; 760M; 1.3B; 同 total 2.7B; 6.7B; 13B; 175B	同 total	每个约 300B	A	同家族参数 scaling 与 few-shot 趋势；不能估 compute-optimal ratio。
2021	Gopher[8]	Dense	44M; 117M; 417M; 1.4B; 7.1B; 同 total 280B	同 total	主模型约 300B	A—	尺寸 scaling 与不同任务收益不均衡。
2022	Chinchilla[7]	Dense	70B	同 total	1.4T	A	与 Gopher 的同计算对照及 compute-optimal allocation。
2022	PaLM[25]	Dense	8B; 62B; 540B	同 total	780B	A—	家族内尺寸趋势与 apparent benchmark thresholds。
2022	OPT[26]	Dense	125M; 350M; 1.3B; 2.7B; 6.7B; 同 total 13B; 30B; 66B; 175B	同 total	主设置约 180B	B+	尺寸覆盖广；须结合 recipe 与 loss 控制。
2022	BLOOM[27]	Dense	560M; 1.1B; 1.7B; 3B; 7.1B; 同 total 176B	同 total	176B 模型约 350B	B	多语言规模观察；尺寸梯度有缺口。
2023	Pythia[21]	Dense	70M; 160M; 410M; 1B; 1.4B; 同 total 2.8B; 6.9B; 12B	同 total	每个约 300B	A+	最佳公开受控 suite 之一，适合训练动力学。
2023	Cerebras-GPT[22]	Dense	111M; 256M; 590M; 1.3B; 2.7B; 同 total 6.7B; 13B	同 total	20 to-kens/parameter	A	公开 Chinchilla-style family。
2023	LLaMA[23]	Dense	7B; 13B; 33B; 65B	同 total	1.0T 或 1.4T	B+	data-rich 小模型可超过旧大模型；不是单轴拟合。
2023	Llama 2[24]	Dense	7B; 13B; 70B	同 total	2T	B+	同代部署梯度；点数不足以稳健拟合指数。
2023	Falcon[28]	Dense	7B; 40B; 180B	同 total	180B 最高约 3.5T	B	data-rich 与 web filtering；token budget 随尺寸变化。
2023	DeepSeek LLM[16]	Dense	7B; 67B	同 total	2T	A—	pilot 外推，并显示数据版本会改变拟合指数。
2023	Mixtral 8x7B[29]	MoE	46.7B	12.9B	未公开	C+	total 与 active compute 必须分开；不是 scaling fit。
2024	OpenELM[30]	Dense	270M; 450M; 1.1B; 3B	同 total	1.5T	B+	同 token budget 的小模型架构比较。
2024	OLMo[31]	Dense	1B; 7B	同 total	约 2T–3T	B+	数据顺序、checkpoints 与训练日志可审计。
2024	Llama 3.1[32]	Dense	8B; 70B; 405B	同 total	超过 15T	B+	现代 data-rich family；不支持 universal coefficients。
2024	Qwen2.5[33]	Dense	0.5B; 1.5B; 3B; 7B; 14B; 32B; 同 total 72B	同 total	18T	B+	现代尺寸梯度，显示 data/recipe 移动 frontier。
2024	Qwen2.5-Coder[34]	Dense	0.5B; 1.5B; 3B; 7B; 14B; 32B	同 total	续训语料 5.5T	B+	代码域梯度；需区分 base pretraining 与 CPT。

下页续

表 11 (续)

年份	家族	架构	Total parameter sizes	Active/token	Pretraining tokens	证据	可支持的判断
2024	Gemma 1[35]	Dense	2B; 7B	同 total	2T; 6T	B	两个点; 说明部署家族不固定 tokens/parameter。
2024	Gemma 2[36]	Dense	2B; 9B; 27B	同 total	2T; 8T; 13T	C+	2B/9B 含 distillation, 反例说明不能只看参数。
2024	DeepSeek-V2[37]	MoE	236B	21B	8.1T	B	分离存储容量与 per-token compute。
2024	DeepSeek-V3[38]	MoE	671B	37B	14.8T	B	大规模 MoE 与计算披露; 单 endpoint 不是 law。
2024	OLMo 2[39]	Dense	1B; 7B; 13B; 32B	同 total	最高 5T-6T	B+	现代 fully open family, 适合数据质量研究。
2025	Qwen3[40]	Dense+ MoE	0.6B; 1.7B; 4B; 8B; 14B; 32B; 30B-MoE; 235B-MoE	Dense 同 total; MoE 3B/22B	约 36T	B+	同代 dense 梯度与 active-parameter MoE 对照。
2025	Llama 4[41]	MoE	Scout 109B; Maverick 400B	均 17B	未以 fit-ready 形式披露	C+	多模态与 distillation 混杂, 只支持 active/total 区分。
2025	Gemma 3[42]	Dense 多模态	多 270M; 1B; 4B; 12B; 27B	同 total	随尺寸变化	C+	部署尺寸梯度; 多模态与 recipe 限制经典比较。
2026	Gemma 4[43]	Dense+ MoE 多模态	E2B; E4B; 12B; 31B; 26B-A4B	A4B 约 4B active	未公开	C	当前 inventory; 缺乏拟合 scaling law 的证据。
2026	DeepSeek-V4[44]	MoE	Flash 284B; Pro 1.6T	13B; 49B	未公开	C	total capacity 增长而 active compute 相对克制; 非 fit-ready。

C 建议的数据记录 Schema 与发车清单

表 12 Scaling experiment 最低记录字段。

字段组	必填内容
模型	total params、non-embedding params、active params、layers、hidden size、heads、experts、context length
数据	raw snapshot、pipeline version、tokenizer hash、actual/unique tokens、epochs、语言与 code mix、dedup ratio
训练	global batch tokens、optimizer、peak/min LR、warmup、schedule、weight decay、seed、precision、gradient clipping
计算	theoretical FLOPs、measured FLOPs、tokens/s、GPU/NPU hours、MFU、失败与重启记录
评估	validation dataset hash、in/OOD loss、bits/byte、benchmark harness、prompt、sampling、污染审计版本
拟合	函数形式、初值、bounds、loss function、异常点策略、run-level bootstrap、holdout frontier、prediction interval
决策	目标规模、预算、 N/D 推荐区间、guardrail、预期 data multiplier、confirmation run 与 go/no-go 结论

发车前检查：

- 1. 三种 compute-optimal 方法是否在目标尺度附近一致？
- 2. 最大 compute holdout 是否落入 prediction interval？
- 3. 目标决策对 E, α, β 的不确定性是否稳健？
- 4. 数据版本、tokenizer、架构或 context 是否发生 regime change？
- 5. OOD/code/repo loss 与 aggregate loss 是否给出一致或可解释的信号？
- 6. 推理量、显存和 latency 是否使 training-optimal 偏离 lifecycle-optimal？

参考文献

- [1] J. Hestness et al.. *Deep Learning Scaling is Predictable, Empirically*. arXiv:1712.00409, 2017.
- [2] J. S. Rosenfeld et al.. *A Constructive Prediction of the Generalization Error Across Scales*. ICLR, 2020.
- [3] J. Kaplan et al.. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [4] Y. Bahri et al.. *Explaining Neural Scaling Laws*. JMLR, 2024.
- [5] E. Caballero et al.. *Broken Neural Scaling Laws*. arXiv:2210.14891, 2022.
- [6] T. B. Brown et al.. *Language Models are Few-Shot Learners*. NeurIPS, 2020.
- [7] J. Hoffmann et al.. *Training Compute-Optimal Large Language Models*. NeurIPS, 2022.
- [8] J. W. Rae et al.. *Scaling Language Models: Methods, Analysis & Insights from Training Gopher*. arXiv:2112.11446, 2021.
- [9] T. Besiroglu et al.. *Chinchilla Scaling: A Replication Attempt*. arXiv:2404.10102, 2024.
- [10] T. Henighan et al.. *Scaling Laws for Autoregressive Generative Modeling*. arXiv:2010.14701, 2020.
- [11] D. Hernandez et al.. *Scaling Laws for Transfer*. arXiv:2102.01293, 2021.
- [12] OpenAI. *GPT-4 Technical Report*. arXiv:2303.08774, 2023.
- [13] J. Wei et al.. *Emergent Abilities of Large Language Models*. TMLR, 2022.
- [14] R. Schaeffer, B. Miranda, and S. Koyejo. *Are Emergent Abilities of Large Language Models a Mirage?*. NeurIPS, 2023.
- [15] N. Muennighoff et al.. *Scaling Data-Constrained Language Models*. arXiv:2305.16264, 2023.
- [16] DeepSeek-AI et al.. *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. arXiv:2401.02954, 2024.
- [17] J. Li et al.. *DataComp-LM: In Search of the Next Generation of Training Sets for Language Models*. arXiv:2406.11794, 2024.
- [18] J. Ye et al.. *Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance*. arXiv:2403.16952, 2024.
- [19] N. Sardana et al.. *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws*. ICML, 2024.
- [20] C. Snell et al.. *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters*. arXiv:2408.03314, 2024.
- [21] S. Biderman et al.. *Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling*. ICML, 2023.
- [22] N. Dey et al.. *Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Pile*. arXiv:2304.03208, 2023.
- [23] H. Touvron et al.. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.
- [24] H. Touvron et al.. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288, 2023.
- [25] A. Chowdhery et al.. *PaLM: Scaling Language Modeling with Pathways*. arXiv:2204.02311, 2022.
- [26] S. Zhang et al.. *OPT: Open Pre-trained Transformer Language Models*. arXiv:2205.01068, 2022.
- [27] T. L. Scao et al.. *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model*. arXiv:2211.05100, 2022.
- [28] E. Almazrouei et al.. *The Falcon Series of Open Language Models*. arXiv:2311.16867, 2023.
- [29] A. Q. Jiang et al.. *Mixtral of Experts*. arXiv:2401.04088, 2024.
- [30] S. Mehta et al.. *OpenELM: An Efficient Language Model Family with Open Training and Inference Framework*. arXiv:2404.14619, 2024.
- [31] D. Groeneveld et al.. *OLMo: Accelerating the Science of Language Models*. arXiv:2402.00838, 2024.
- [32] Meta AI. *Introducing Llama 3.1: Our Most Capable Models to Date*. Official release, 2024.
- [33] Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115, 2024.
- [34] B. Hui et al.. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186, 2024.
- [35] Gemma Team. *Gemma: Open Models Based on Gemini Research and Technology*. arXiv:2403.08295, 2024.
- [36] Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size*. arXiv:2408.00118, 2024.
- [37] DeepSeek-AI et al.. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv:2405.04434, 2024.
- [38] DeepSeek-AI et al.. *DeepSeek-V3 Technical Report*. arXiv:2412.19437, 2024.
- [39] Allen Institute for AI. *OLMo 2: The Best Fully Open Language Model to Date*. Official release, 2024.
- [40] Qwen Team. *Qwen3: Think Deeper, Act Faster*. Official technical blog, 2025.
- [41] Meta AI. *The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation*. Official release, 2025.
- [42] Google. *Gemma Models Documentation*. Official documentation; accessed 2026-08-07, 2025.
- [43] Google. *Gemma Models Documentation: Current Model Inventory*. Official documentation; accessed 2026-08-07, 2026.
- [44] DeepSeek. *DeepSeek-V4 Official Release*. Official release, 2026.